Order Shipment Analysis

Demetri Bichara

2026-11-07

Contents

Part 1 - Goal A: **1**
Solution . . . 1
Methodology . . . 4

Part 1 - Goal B: **7**
Response . . . 7

Part 1 - Goal A:

Provide the following information for any instance:

1. **Order #**
2. **Item Name**
3. **Quantity**
4. **Total value** (price x quantity)

Solution

```
unshipped <- orders |>
  left_join(shipments, by = 'id', relationship = 'one-to-one') |>
  left_join(items, by = 'item_id', relationship = 'many-to-one') |>
  mutate(
    why_unshipped = case_when(
      is.na(status) ~ "No shipment record",
      status == "Pending" ~ "Not yet shipped",
      TRUE ~ NA),
    total_value = price * quantity) |>
  filter(!is.na(why_unshipped)) |>
  select(order_id, id, name, quantity, total_value, why_unshipped) |>
  arrange(order_id, id)
```

```
unshipped
```

order_id	id	name	quantity	total_value	why_unshipped
65	275	Network Attached Storage	20	8000	No shipment record
70	292	Monitor 27-inch	8	2240	No shipment record
70	294	External SSD 1TB	2	240	No shipment record
70	297	Extended Warranty 3yr	5	1000	No shipment record
120	541	DisplayPort Cable	34	476	No shipment record

order_id	id	name	quantity	total_value	why_unshipped
125	570	Motherboard - Micro ATX	1	150	No shipment record
140	637	Wireless Keyboard	33	1485	No shipment record
162	750	GPU - Mid Range	25	13750	No shipment record
166	765	Network Attached Storage	5	2000	No shipment record
186	860	Wi-Fi USB Adapter	14	350	No shipment record
297	1352	Motherboard - Mini ITX	17	3060	No shipment record
300	1367	RAM - 8GB	20	3000	No shipment record
319	1451	PC Case - Mid Tower	24	2160	No shipment record
322	1462	Mini PC	31	13950	No shipment record
333	1499	Wireless Mouse	17	510	No shipment record
337	1526	Chromebook	11	3850	No shipment record
374	1680	Laptop Stand	9	315	No shipment record
411	1812	Gaming Mouse	16	960	No shipment record
444	1960	PC Case - Full Tower	19	3040	No shipment record
483	2137	Bluetooth Speaker	28	2520	Not yet shipped
483	2138	RAM - 32GB	17	9350	Not yet shipped
483	2139	Case Fan 120mm	17	204	Not yet shipped
483	2140	USB-C Cable 1m	4	40	Not yet shipped
483	2142	Motherboard - Mini ITX	26	4680	Not yet shipped
484	2144	SD Card 128GB	13	260	Not yet shipped
484	2145	Smartphone	9	7200	Not yet shipped
484	2146	SSD 500GB	3	210	Not yet shipped
484	2147	Monitor Arm	27	2025	Not yet shipped
484	2148	Power Supply 1000W	32	5760	Not yet shipped
487	2152	SD Card 128GB	2	40	Not yet shipped
487	2153	Laptop Bag	24	1320	Not yet shipped
487	2154	Surge Protector	6	150	Not yet shipped
487	2155	Motherboard - Mini ITX	19	3420	Not yet shipped
487	2156	USB Flash Drive 64GB	4	48	No shipment record
487	2157	HDD 8TB	29	4640	Not yet shipped
487	2158	CPU - Mid Range	22	7040	Not yet shipped
487	2159	USB-C Cable 1m	28	280	Not yet shipped
488	2161	GPU - Mid Range	32	17600	Not yet shipped
491	2168	Ethernet Cable 10m	30	450	Not yet shipped
491	2169	Operating System License	22	3080	Not yet shipped
491	2170	Motherboard - ATX	22	4840	Not yet shipped
491	2171	RAM - 32GB	35	19250	Not yet shipped
491	2172	Flatbed Scanner	13	1820	Not yet shipped
491	2173	GPU - Entry Level	25	7000	Not yet shipped
491	2174	Power Bank 20000mAh	14	630	Not yet shipped
494	2188	Ethernet Cable 2m	7	42	Not yet shipped
494	2189	Motherboard - Micro ATX	20	3000	Not yet shipped
495	2190	SD Card 128GB	14	280	Not yet shipped
495	2191	HDD 8TB	30	4800	Not yet shipped
495	2192	GPU - Mid Range	25	13750	Not yet shipped
495	2193	Monitor 32-inch 4K	26	12480	Not yet shipped
495	2194	Wireless Keyboard	18	810	Not yet shipped
495	2195	Ethernet Cable 2m	12	72	Not yet shipped
496	2197	Tablet 12-inch	26	19500	Not yet shipped
496	2199	Ergonomic Mouse	10	700	Not yet shipped
497	2202	Wireless Keyboard	16	720	Not yet shipped
497	2203	HDD 8TB	20	3200	Not yet shipped

order_id	id	name	quantity	total_value	why_unshipped
497	2204	HDMI Cable 2m	2	18	Not yet shipped
498	2205	RAM - 32GB	11	6050	Not yet shipped
498	2206	Wireless Access Point	3	420	Not yet shipped
498	2207	Monitor 27-inch	11	3080	Not yet shipped
498	2208	SSD 4TB	31	11160	Not yet shipped
498	2209	All-in-One PC	24	30000	Not yet shipped
498	2210	RAM - 16GB	10	3000	Not yet shipped
498	2211	Laptop Stand	20	700	Not yet shipped
498	2212	USB-C Cable 1m	33	330	Not yet shipped
499	2213	SD Card 128GB	30	600	Not yet shipped
499	2214	External HDD 4TB	10	1050	Not yet shipped
499	2215	Chromebook	24	8400	Not yet shipped
500	2216	GPU - Mid Range	20	11000	Not yet shipped
500	2217	Laptop Bag	18	990	Not yet shipped

```
unshipped |>
  summarize(
    missing_prices = sum(is.na(total_value))
    )
```

missing_prices
0

```
unshipped |>
  summarize(
    affected_orders = n_distinct(order_id),
    unshipped_lines = n(),
    total_unshipped_value = sum(total_value)
  )
```

affected_orders	unshipped_lines	total_unshipped_value
29	71	300545

```
unshipped |>
  group_by(why_unshipped) |>
  summarize(
    affected_orders = n_distinct(order_id),
    unshipped_lines = n(),
    total_unshipped_value = sum(total_value),
    .groups = "drop"
  )
```

why_unshipped	affected_orders	unshipped_lines	total_unshipped_value
No shipment record	18	20	60584
Not yet shipped	12	51	239961

To ID unshipped order lines, I first established the relationships between the orders, shipments, and items tables. The id column was found to be the most probable order-line identifier shared between orders and

shipments. I ID'd 20 order lines that did not have a corresponding shipment record, and 51 shipment statuses that are Pending. A Complete shipment is assumed to fulfill the entire ordered quantity, since shipments don't specify shipment-level quantities. Lastly, I calculated total unshipped value for each unshipped order line via price * quantity.

Methodology

First, we need to know how to map 1 order to their corresponding shipments. The most probable path is a left joining of the shipments CSV on the orders CSV, which preserves every order line item (including the order line items without corresponding shipments).

From inspecting the data in the two CSV's with RStudio, I can see the orders.id column has the same range as the shipments.id column. Due to this similarity, I think that the best way to left join the two tables is by using the `id` column for equality checking. If correct, the relationship for the two `id` columns is 1:0-1 (0 in the case of a missed shipment).

We can do the same duplicate-checking test to start to assert the truthfulness of the described many-to-one relationship for the `item_id` column across the two tables orders and items.

Checking for any duplicate values in the `id` column within both tables, which would break the proposed 1:0-1 mapping.

1:0-1 test-A:

```
any(duplicated(shipments$id)) # should be FALSE
```

```
## [1] FALSE
```

```
any(duplicated(orders$id)) # should be FALSE
```

```
## [1] FALSE
```

N:1 test-A:

```
any(duplicated(orders$item_id)) # should be TRUE (many-to-one)
```

```
## [1] TRUE
```

```
any(duplicated(items$item_id)) # should be FALSE
```

```
## [1] FALSE
```

Since there aren't duplicates in the `id` column of either the shipments CSV or the orders CSV, we can further test my proposed relationship between the two tables by checking whether all `id` values in the shipment CSV exist in the `id` column of the orders CSV with an anti join. This will not fail on missed shipments since anti_join(shipments, orders) only tests whether shipment records have corresponding order lines. We can do the same check with the `item_id` column across the tables orders and items.

1:0-1 test-B:

```
shipments |> anti_join(orders, by = 'id')
```

shipment_id	id	status	ship_date

0 rows missing a corresponding id!

N:1 test-B:

```
orders |> anti_join(items, by = 'item_id')
```

id	order_id	order_date	item_id	company	quantity

0 rows missing a corresponding id!

Since all testing passed so far, we can further test the understood relationships by setting an expected relationship during the joining process.

```
orders |>
  left_join(shipments, by = 'id', relationship = 'one-to-one') |>
  left_join(items, by = 'item_id', relationship = 'many-to-one')
```

The joins executed without error, meaning matches during the join did not violate the declared relationships (wouldn't catch a missing shipment, okay because the cardinality is 1:0-1). The checks collectively provide strong evidence to move forward with the understanding.

Now that we are confident in the relationships, we can start to investigate missing shipments by reversing the anti-join direction from '1:0-1 test-B'. This would show all order line items that did not have a corresponding shipment. Importantly, this would not catch all unshipped order lines by itself, since the corresponding status might not be 'Complete'. Moreover, since the shipments table doesn't contain a quantity column, we will assume that a shipment line entry / row with the 'Complete' status indicates the entire corresponding quantity was sent.

```
orders |> anti_join(shipments, by = 'id')
```

id	order_id	order_date	item_id	company	quantity
275	65	2025-12-16	73	Qualcomm	20
292	70	2025-12-17	47	Starbucks	8
294	70	2025-12-17	24	Starbucks	2
297	70	2025-12-17	98	Starbucks	5
541	120	2026-01-26	79	Intel	34
570	125	2026-01-30	35	Microsoft	1
637	140	2026-02-09	52	Apple	33
750	162	2026-02-23	32	Delta Air Lines	25
765	166	2026-02-25	73	Qualcomm	5
860	186	2026-03-14	71	FedEx	14
1352	297	2026-06-04	36	Dell Technologies	17
1367	300	2026-06-07	14	Humana	20
1451	319	2026-06-19	40	Berkshire Hathaway	24
1462	322	2026-06-20	8	IBM	31
1499	333	2026-06-30	54	UPS	17
1526	337	2026-07-01	6	Home Depot	11
1680	374	2026-08-11	84	Honeywell	9
1812	411	2026-09-05	55	Adobe	16
1960	444	2026-09-29	41	Walmart	19
2156	487	2026-10-25	26	Amazon	4

Checks for orders without matching shipment records: matched 20 order line items

```r
unique(shipments$status)
```

```
## [1] "Complete" "Pending"
```

2 possible values for a status entry {'Complete', 'Pending'} (no na). The literal meaning of `Pending` might be loose: if its been pending for 9 months, is that an anomaly or a true shipping time? The most recent date in the shipments table's `ship_date` column is 11-02 of this year (2026), so we can use that point for detecting zombie shipments with an aging threshold. However, for purposes of this beta, we can further analysis knowing only 'pending'<->'not yet shipped'.

As to verify the data quality before further analysis, we can also make sure no shipment dates precede their order dates.

```r
orders |>
  left_join(shipments, by = 'id') |>
  filter(!is.na(ship_date), ship_date < order_date)
```

id	order_id	order_date	item_id	company	quantity	shipment_id	status	ship_date

Matches 0 orders who have an earlier corresponding shipment record.

Same type on the two `*_date` fields makes the comparison possible, so we should test that:

```r
class(orders$order_date)
```

```
## [1] "Date"
```

```r
class(shipments$ship_date)
```

```
## [1] "Date"
```

While {"Complete", "Pending"} are the only two values in the `status` column, it's worth it to check for shipments marked as `Complete` without it's corresponding evidence (the ship date). While a missing ship date doesn't automatically mean a false status entry, we do not have access to the underlying business logic generating the DB, so we can mark the results from all shipments who have a status = 'Complete' and no ship date.

```r
shipments |>
  filter(
    (status == "Complete" & is.na(ship_date)) |
      (status == "Pending" & !is.na(ship_date)) # check the reverse also to make
    #                                            sure business logic
    #                                             understanding is right
    #                                 (pending shipments cannot have dates)
  )
```

shipment_id	id	status	ship_date

No such case! Moreover, we have to handle any missing shipment_id rows as to handle 'Pending' for ghost shipments

```r
shipments |>
  filter(is.na(shipment_id)) |>
  count(status)
```

status	n
Pending	51

Matched 51 shipment record whose status is pending without a shipment ID (they shouldn't have it). Didn't catch any Complete order line items without a shipment ID.

Assert we found all pending records.

```
shipments |>
  filter(status == "Pending") |>
  summarize(n = n())
```

n
51

Since both tests display 51 records, we are more confident in the understanding of the business logic.

With a strong validation / understanding of the data via the first phase, we can now finally get the raw unshipped order line item numbers and handle the observed cases!

Part 1 - Goal B:

Describe how you would implement a process using to alert you of instances when orders didn't ship completely. How would you track it? How would you automate it?

Response

Objective

I'd make a daily or bi-daily monitoring process that keeps track of marked order line items and their order_ids, and configure the appropriate teams messaging to automatically receive notices for when an order exceeds an agreed shipping timeframe.

1. Detect incomplete orders

The process would refresh the orders, shipments, and items datasets daily. Using the relationships established in Part 1, it would join the tables and classify each order line as Complete, Pending, or Missing Shipment Record. If the refresh fails, I'd notify the software maintainer.

An order would be considered fully shipped only when all its corresponding order lines have a Complete status.

2. Track outstanding shipments

I would program the script to maintain a persistent table (periodically saving/overwriting backups to disk in case of a run-time crash) containing the order ID, order-line ID, customer, item, quantity, unshipped value, order date, current shipment status, and why the shipment was incomplete for each marked unshipped shipment line item.

I would include more fields for recording when the exception was first detected, how long it has been since, whether an alert has been sent, and if / when it's been resolved. We can remove records from the tracking DB when the issue has been resolved for more than 30 days. We can also use the described fields to know when to re-notify a team of the issue.

Missing shipment records allude to incomplete order lines, so an order should only be classified as fully shipped when every associated order line has a Complete shipment status.

```
order_completion <- orders |>
  left_join(
    shipments,
    by = "id",
    relationship = "one-to-one"
  ) |>
  group_by(order_id) |>
  summarize(
    total_lines = n(),
    incomplete_lines = sum(
      is.na(status) | status != "Complete"
    ),
    fully_shipped = incomplete_lines == 0,
    .groups = "drop"
  )

order_completion |>
  filter(!fully_shipped)
```

order_id	total_lines	incomplete_lines	fully_shipped
65	6	1	FALSE
70	8	3	FALSE
120	6	1	FALSE
125	8	1	FALSE
140	4	1	FALSE
162	7	1	FALSE
166	8	1	FALSE
186	7	1	FALSE
297	2	1	FALSE
300	7	1	FALSE
319	4	1	FALSE
322	3	1	FALSE
333	7	1	FALSE
337	8	1	FALSE
374	1	1	FALSE
411	5	1	FALSE
444	5	1	FALSE
483	7	5	FALSE
484	5	5	FALSE
487	8	8	FALSE
488	2	1	FALSE
491	7	7	FALSE
494	2	2	FALSE
495	6	6	FALSE
496	6	2	FALSE
497	3	3	FALSE
498	8	8	FALSE
499	3	3	FALSE
500	2	2	FALSE

```
joined <- orders |>
  left_join(
```

```
    shipments,
    by = "id",
    relationship = "one-to-one"
  )

joined |>
  mutate(
    incomplete = is.na(status) | status != "Complete"
  ) |>
  summarize(
    total_lines = n(),
    incomplete_lines = sum(incomplete),
    affected_orders = n_distinct(order_id[incomplete])
  )
```

total_lines	incomplete_lines	affected_orders
2217	71	29

```
joined |>
  count(status)
```

status	n
Complete	2146
Pending	51
NA	20

```
order_completion |>
  filter(!fully_shipped) |>
  summarize(
    affected_orders = n_distinct(order_id)
  )
```

affected_orders
29

This would allow the company to track individual missed shipments across multiple days and can be used by a non-technical person.

3. Define alert conditions

Every incomplete order doesn't need to generate an immediate alert, because the company intentionally fulfills orders at different times.

I would work with the operations team to establish an acceptable shipping timeframe. For example, orders remaining incomplete after two business days could generate an initial alert, while those outstanding for five business days could be escalated. Alerts would include the order number, affected items, and days outstanding.

4. Automate the process

I would refactor the existing R analysis into a scheduled script. The script would retrieve updated data, run validation checks, identify incomplete orders, and update the exception tracker.

An email could summarize newly overdue orders and previously flagged orders that remain unresolved. Previously reported exceptions would not trigger duplicate initial alerts, and resolved orders would automatically be marked as closed. We can add a false-positive feedback button in notifications to know if the process needs improvement, and another periodically-sent form to report missed shipments the process didn't catch.

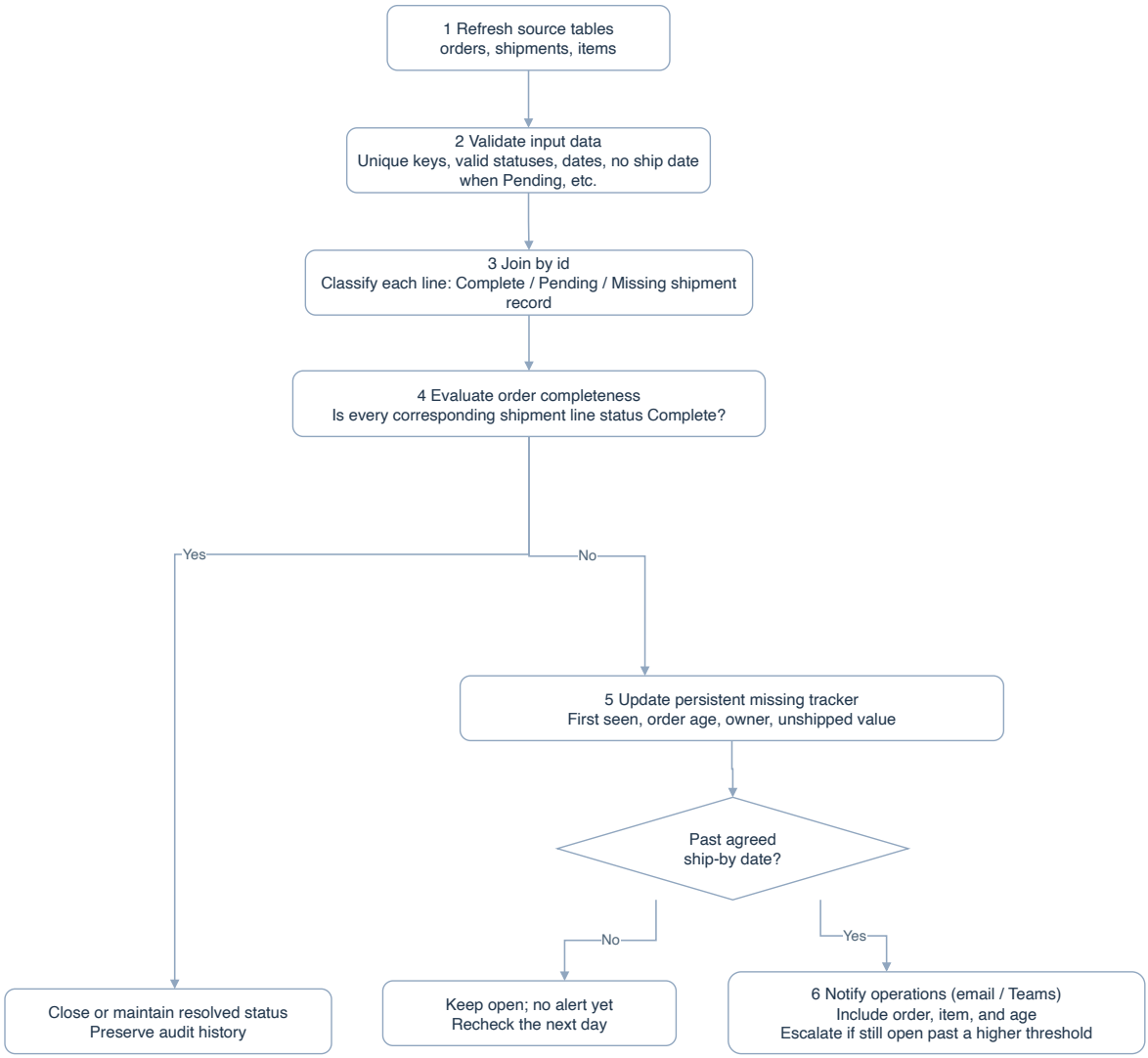


Figure 1: Workflow Visualized